SL Coursework 2

Daniel Pulido Espinosa

December 2025

1 Part I

2 Part II

Tables 1 and 2 present the empirical generalisation errors of Laplacian Interpolation (LI) and Laplacian kernel interpolation (LKI), respectively. As illustrated in both tables, the error decreases as the number of known labels per class grows. This alligns with the ideas of learning theory, as a higher volume of labelled data helps the model define a more precise decision boundary.

More notably, the error also decreases as the total number of data points per label increases. This could seem counter-intuitive, as a decrease in the ratio of labelled data might suggest a loss of information. However, these new unlabelled data points give information about the underlying data manifold. Therefore, when constructing the 3-NN graph, the nearest neighbours will be closer in the feature space. This reduces the probability of there being edges connecting points of different labels, assuming that points within the same cluster are likely to be in the same class. Consequently, the geometrical information gained from the new unlabelled data compensates for the scarcity of labels, improving the model's predictive performance. This highlights the strength of semi-supervised learning, showing how the model can gain valuable information from unlabelled data.

Another significant result is the general improvement in performance of LKI over LI. Theoretically, both algorithms solve the same optimization problem - LI through the primal form and LKI through the dual form via the Representer Theorem. However, the LKI approach is more numerically stable, as it avoids matrix inversion, by using the Moore-Penrose pseudoinverse instead. Moreover, during the computation of the pseudoinverse, a slight implicit regularisation is performed, discarding near-zero eigenvalues. This regularisation enhances the smoothness of the solution and prevents the "spiking" effect common in LI, where label influence fails to propagate beyond the immediate neighborhood of a labeled node. As evidenced by the lower standard deviations in Table 2, LKI produces more reliable predictions, with lower variance.

In conclusion, this exercise highlights the power of semi-supervised learning, demostrating how the geometrical information given by unlabelled data can be of use to make predictions of their labels. Furthermore, it showcases the advantages of using the pseudoinverse instead of the normal inverse, both in terms of numerical instability and regularisation, which makes LKI produce better predictions.

		# of known labels (per class)				
		1	2	4	8	16
# data points per label	50	0.162 ± 0.105	0.129 ± 0.092	0.078 ± 0.048	0.055 ± 0.029	0.039 ± 0.019
	100	0.057 ± 0.034	0.045 ± 0.021	0.042 ± 0.014	0.041 ± 0.015	0.036 ± 0.015
	200	0.083 ± 0.095	0.027 ± 0.016	0.019 ± 0.007	0.023 ± 0.008	0.019 ± 0.007
	400	0.133 ± 0.188	0.021 ± 0.014	0.013 ± 0.002	0.012 ± 0.002	0.01 ± 0.002

Table 1: Errors and standard deviations for semi-supervised learning via Laplacian interpolation

		# of known labels (per class)				
		1	2	4	8	16
# data points per label	50	0.104 ± 0.061	0.067 ± 0.032	0.061 ± 0.035	0.04 ± 0.009	0.037 ± 0.016
	100	0.046 ± 0.019	0.043 ± 0.02	0.04 ± 0.013	0.04 ± 0.014	0.032 ± 0.009
	200	0.027 ± 0.027	0.02 ± 0.005	0.017 ± 0.005	0.02 ± 0.005	0.016 ± 0.005
	400	0.018 ± 0.027	0.013 ± 0.004	0.012 ± 0.002	0.011 ± 0.001	0.01 ± 0.002

Table 2: Errors and standard deviations for semi-supervised learning via Laplacian kernel interpolation

3 Part III

a) The following figures show the plots of the sample complexity vs input dimension for the perceptron, winnow, least squares and 1-NN algorithms:

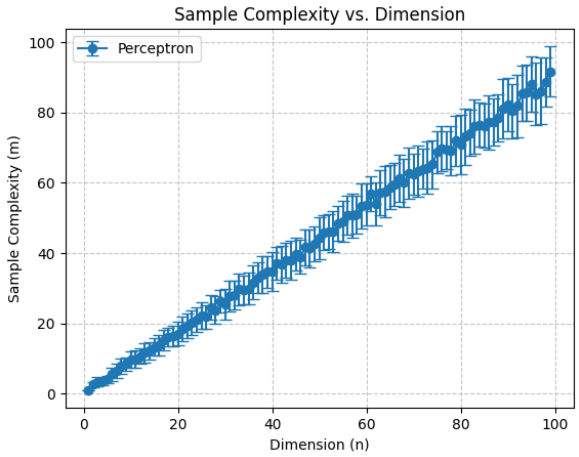


Figure 1: Sample complexity of Perceptron

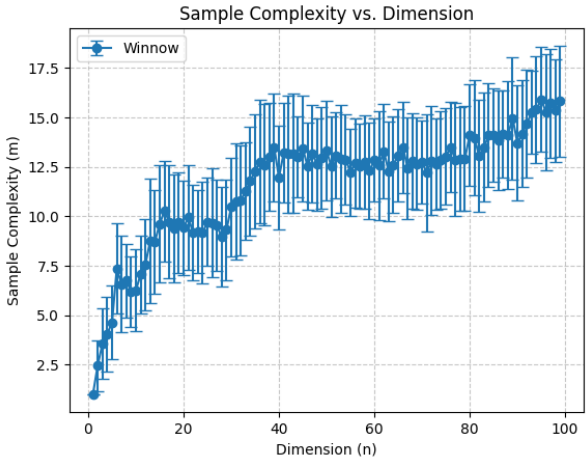


Figure 2: Sample complexity of Winnow

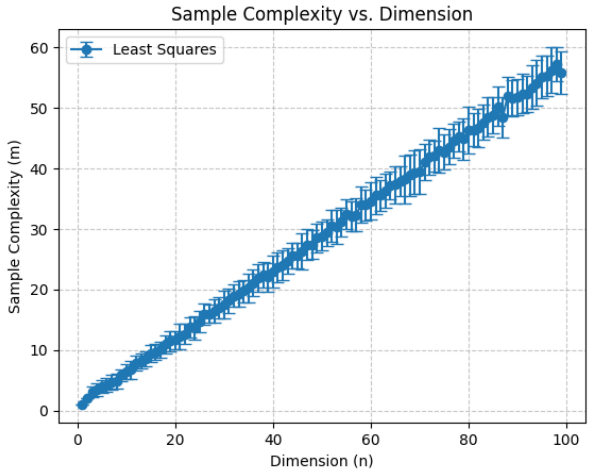


Figure 3: Sample complexity of least squares

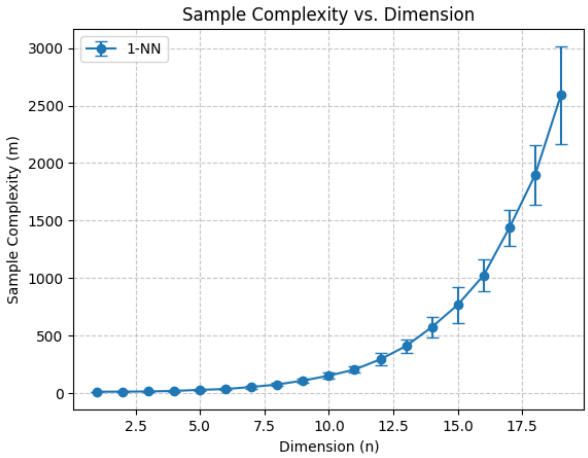


Figure 4: Sample complexity of 1-NN

b) Our implementation estimates sample complexity using a Monte Carlo approximation to avoid the exponential cost of evaluating all 2^{nm} possible data sequences. For each input dimension n, we conduct 50 independent trials. In each trial, we iteratively increase the training set size m (using a defined step size) until the empirical generalization error falls below the threshold $\epsilon = 0.1$. The step size is equal to 1 for all algorithms except for 1-NN, due to its high computational cost.

To ensure the estimate is robust, independence between trials is guaranteed through the following ways:

- For each value of m, a new training set of size m is drawn uniformly at random from $\{-1, 1\}^n$.
- The generalisation error is estimated using an independent, fixed- test dataset for each trial and value of m.

Finally, the mean and standard deviation of the results of those 50 trials are computed. The mean represents our estimate of $\mathcal{C}(\mathcal{A})$ for that specific input dimension n.

Our implementation trades-off computational cost for estimator bias. By sampling a finite number of trials and test patterns, we reduce the problem into one that can be solved in polynomial time. However, we incur in several biases:

- **Early Exit Bias:** As the algorithm exits as soon as a single trial's error drops below 0.1, it is susceptible to being tricked by specific train datasets that represent the target function better than the average case, or test datasets which are overly similar to the one used to train the model. These outliers reduce the final value of the estimator, making it downwardly biased. As the minimum m is selected in each trial, the presence of an outlier dataset which makes the predictions harder is negligible, as for the next value of m a new dataset will be generated, which will probably not be an outlier.
- **Step-size quantification:** Using a step size greater than 1, as done with 1-NN, introduces a small upward bias. For example, with a step size of 10, the true crossing point may occur at m=96, but it will be detected at the next step which is m=101. It also increases the variance, as the possible values for m have a greater separation between each other.

c) The sample complexity for the four algorithms grows in the following way:

- **Perceptron:** figure 1 displays a clear linear relationship between input dimension and sample complexity. Therefore, $m = \Theta(n)$.
- **Winnow:** figure 1 displays a sublinear relationship between input dimension and sample complexity, similar to a logarithmic curve. Therefore, $m = \Theta(log(n))$-
- **Least Squares:** figure 3 displays a clear linear relationship between input dimension and sample complexity. Therefore, $m = \Theta(n)$. However, the slope is lower than in the perceptron algorithm.
- **Perceptron:** figure 1 displays an exponential relationship between input dimension and sample complexity. Therefore, $m = \Theta(e^n)$.

In conclusion, Winnow seems to perform best in this scenario, followed by Least Squares and Perceptron, with 1-NN being the worst algorithm.